

Proyecto Procesamiento de Texto con LLM y Modelo de Machine Learning (Fase 1 y Fase 2)

1. Definición del proyecto

El proyecto consistirá en el desarrollo de un sistema de procesamiento de texto orquestado mediante **n8n y/o Apache Airflow**.

El sistema procesará textos mediante modelos LLM y un modelo de Machine Learning para:

- Ingerir textos en formato `.txt` o registros procedentes de un dataset.
- Ejecutar un pipeline de procesamiento lingüístico con LLM.
- Extraer entidades relevantes del texto.
- Normalizar entidades para agrupar conceptos equivalentes.
- Etiquetar cada texto según una categoría funcional, emocional o clínica.
- Calcular un score de ansiedad u otra métrica definida por el proyecto.
- Generar un dataset enriquecido para entrenar un modelo de Machine Learning.
- Entrenar y versionar un modelo de Machine Learning.
- Utilizar el modelo entrenado para realizar predicciones sobre nuevos textos.
- Valorar la predicción en una segunda fase de prueba y evaluación.

El sistema se dividirá en dos fases principales:

- **Fase 1:** procesamiento inicial de textos, enriquecimiento con LLM, generación de etiquetas, cálculo de scores, creación del dataset y entrenamiento del modelo.
- **Fase 2:** uso del modelo entrenado para predecir el estado o categoría de nuevos textos y registrar la valoración obtenida.

La coordinación del flujo se realizará mediante un **workflow** definido en n8n y/o Airflow. Cada etapa podrá implementarse como una tarea del workflow, un nodo de n8n, una función Python o un microservicio invocado por HTTP. La trazabilidad se mantendrá mediante un identificador único de solicitud o de texto.

2. Requisitos Técnicos

Arquitectura basada en workflow

No se debe utilizar Kafka. La comunicación y coordinación entre etapas se hará mediante:

- **n8n**, para flujos visuales, integraciones HTTP, webhooks y automatización de pasos.
- **Apache Airflow**, para DAGs programados, dependencias entre tareas, reintentos, trazabilidad y ejecución batch.
- Llamadas HTTP internas entre servicios, cuando una etapa se implemente como microservicio.
- Ejecución directa de scripts Python desde Airflow o desde contenedores auxiliares.
- Persistencia de estados intermedios en Postgres.
- Almacenamiento de artefactos en minIO/S3.

Cada texto procesado deberá arrastrar durante todo el pipeline un identificador único, por ejemplo `GUID_Solicitud` o `GUID_Texto`, que permita reconstruir el flujo completo.

Criterios de uso de n8n y Airflow

Se podrá elegir una de estas tres opciones:

1. Solo n8n

- Adecuado para flujos sencillos, visuales y orientados a webhooks.
- Recomendado si el pipeline se ejecuta a demanda desde una petición HTTP.
- Puede invocar APIs internas, servicios LLM, scripts y endpoints de predicción.

2. Solo Airflow

- Adecuado para procesos batch, entrenamiento programado y pipelines con dependencias claras.
- Recomendado para construir datasets, entrenar modelos y ejecutar evaluaciones periódicas.
- Permite definir DAGs versionados en Python.

3. n8n + Airflow

- n8n recibe la petición, valida datos y lanza el flujo.
- Airflow ejecuta el pipeline pesado de procesamiento, entrenamiento o evaluación.
- n8n puede consultar resultados y devolverlos al usuario.

Contenedores Docker Minimmos Recomendados

1. n8n Workflow Service

- **Función:**
 - Recibir eventos de entrada mediante Webhook.
 - Ejecutar llamadas HTTP a los servicios del pipeline.
 - Gestionar ramas simples del flujo.
 - Lanzar un DAG de Airflow cuando sea necesario.
 - Notificar finalización o error.
- **Tecnología:** n8n.
- **Uso recomendado:** flujos de Fase 2, recepción de solicitudes, integración con APIs y notificaciones.

3. Airflow Scheduler / Webserver / Workers

- **Función:**
 - Definir y ejecutar DAGs del pipeline.
 - Coordinar tareas de preprocesamiento, LLM, dataset, entrenamiento y evaluación.
 - Gestionar dependencias entre tareas.
 - Ejecutar reintentos automáticos.
 - Registrar logs por tarea.
- **Tecnología:** Apache Airflow.
- **Uso recomendado:** Fase 1 completa, entrenamiento, evaluación y procesamiento batch.

4. Postgres

- **Función:**
 - Almacenar solicitudes.
 - Guardar textos originales y preprocesados.
 - Registrar entidades, etiquetas, scores, predicciones y valoraciones.
 - Registrar estado de cada paso del workflow.

5. minIO / S3

- **Función:**
 - Almacenar archivos `.txt` originales.
 - Guardar datasets generados.
 - Guardar modelos entrenados y artefactos de evaluación.

Workflows o DAGs recomendados

Si se usa Airflow, estos workflows se implementarán como DAGs:

- `dag_text_ingestion`
- `dag_llm_enrichment`
- `dag_model_training`
- `dag_prediction_phase_2`
- `dag_evaluation`

Si se usa n8n, estos workflows se implementarán como flujos visuales con nodos:

- Webhook de entrada.
- Nodo HTTP Request para llamar a servicios Python/LLM.
- Nodo Postgres para actualizar estados.
- Nodo S3/minIO para recuperar o guardar artefactos.
- Nodo IF/Switch para bifurcaciones.
- Nodo Error Trigger para gestión de fallos.

Métricas de rendimiento

Cada etapa del workflow debe medir:

- Tiempo de procesamiento por tarea.
- Latencia end-to-end.
- Número de textos procesados.
- Throughput por minuto o por ejecución.
- Número de errores por tarea.
- Número de reintentos.
- Tiempo medio de respuesta del LLM.
- Tiempo de entrenamiento del modelo.
- Métricas del modelo: accuracy, precision, recall, F1-score o métrica equivalente.

3. Dataset

Dataset base: textos en formato `.txt` o tabla estructurada con los siguientes campos mínimos:

- `id`
- `origen`

- texto
- entidades
- entidades_normalizadas
- etiqueta
- score_ansiedad
- prediccion_entrenada

Se podrán utilizar dos tipos de registros:

- **Dataset:** textos reales o previamente recopilados.
- **Simulación:** textos creados para probar el comportamiento del pipeline.

Ejemplo de registros de Fase 1:

ID	Origen	Texto	Entidades	Entidades normalizadas	Etiquetado
1	Dataset	Tengo miedo	[miedo]	[ansiedad]	C2
2	Simulación	Miedo a...	[cuesta respirar, cansado]	[disnea, fatiga]	C2
3	Dataset	Miedo a...	[cuesta respirar, cansado]	[disnea, fatiga]	C2

Ejemplo de registros de Fase 2:

ID	Texto	Predicción prueba	Valoración
2	Siento alivio	calma	9/10
3	Siento alivio	calma	9/10
4	Siento alivio	calma	9/10

Todos los equipos deben usar la misma estructura mínima de datos para facilitar la evaluación y comparación de resultados.

4. Orquestación

La orquestación del sistema se realizará mediante **n8n y/o Airflow**.

El orquestador no procesará directamente el texto. Su responsabilidad será coordinar el workflow, lanzar tareas, comprobar resultados, registrar estados, gestionar errores y decidir si el flujo continúa hacia entrenamiento, predicción o evaluación.

El orquestador sería **el componente que controla qué tarea se ejecuta después de cada paso del procesamiento del texto**.

4.1. Flujo Conceptual Recomendado

1. Cliente -> API Gateway de ingesta -> guarda solicitud en Postgres y texto original en minIO/S3 -> lanza workflow de n8n o DAG de Airflow.
2. n8n / Airflow -> crea contexto de ejecución -> arrastra `GUID_Solicitud` y `GUID_Texto` -> actualiza estado inicial.
3. Preprocesamiento Python -> limpia y normaliza el texto -> guarda resultado en Postgres -> devuelve estado `TEXTO_PREPROCESADO`.
4. LLM Entity Extraction -> recibe el texto preprocesado -> extrae entidades -> guarda entidades en Postgres -> devuelve estado `ENTIDADES_EXTRAIDAS`.
5. LLM Normalization -> normaliza entidades -> guarda entidades normalizadas -> devuelve estado `ENTIDADES_NORMALIZADAS`.
6. LLM Labeling -> etiqueta el texto -> guarda la etiqueta -> devuelve estado `ETIQUETADO`.
7. Anxiety Score Service -> calcula el score -> guarda score -> devuelve estado `SCORE_CALCULADO`.
8. Dataset Builder -> genera variables y dataset entrenable -> guarda dataset en minIO/S3 -> devuelve estado `DATASET_GENERADO`.
9. ML Training Service -> entrena el modelo -> guarda modelo y métricas en minIO/S3 -> devuelve estado `MODELO_ENTRENADO`.
10. Cliente / API Gateway Fase 2 -> envía nuevo texto -> lanza workflow de predicción.
11. ML Prediction Service -> carga el modelo entrenado -> genera predicción -> guarda predicción en Postgres.
12. Evaluation Service -> calcula valoración de la predicción -> guarda valoración -> marca el flujo como `COMPLETADA`.

13. Cliente -> API Gateway de consulta -> recibe metadatos, predicción, valoración y estado final.

Observaciones:

- A lo largo de todo el pipeline se arrastra el valor del `GUID_Texto` o `GUID_Solicitud`.
- Cada tarea debe registrar `timestamp_inicio`, `timestamp_fin`, `service_name`, `status` y `payload_resultado`.
- Los errores deben registrarse en Postgres y en el sistema de logs de n8n/Airflow.
- No hay publicación ni consumo de mensajes Kafka.

4.3. Tablas

```
CREATE TABLE Entrevista (
    GUID_Entrevista VARCHAR(255) PRIMARY KEY,
    URL_Texto_Original VARCHAR(255),
    URL_Dataset_Generado VARCHAR(255),
    URL_Modelo_Entrenado VARCHAR(255),
    Inicio_Solicitud TIMESTAMP,
    Fin_Solicitud TIMESTAMP,
    Inicio_Preprocesamiento TIMESTAMP,
    Fin_Preprocesamiento TIMESTAMP,
    Inicio_Extraccion_Entidades TIMESTAMP,
    Fin_Extraccion_Entidades TIMESTAMP,
    Inicio_Normalizacion TIMESTAMP,
    Fin_Normalizacion TIMESTAMP,
    Inicio_Etiquetado TIMESTAMP,
    Fin_Etiquetado TIMESTAMP,
    Inicio_Score TIMESTAMP,
    Fin_Score TIMESTAMP,
    Inicio_Entrenamiento TIMESTAMP,
    Fin_Entrenamiento TIMESTAMP,
    Motor_Workflow VARCHAR(50),
    Workflow_Id VARCHAR(255),
    Estado VARCHAR(50)
);
```

5. Almacenamiento

Para el almacenamiento de ficheros y artefactos se utilizará minIO, compatible con Amazon S3.

Habrà que crear un contenedor con ese servicio.

Consideraciones para el almacenamiento:

- Dentro del bucket de S3 debe haber carpetas separadas para:

- Textos originales.
- Datasets generados.

6. Entregables

Cada grupo creará un repositorio en GitHub con acceso de lectura para el profesor. El contenido mínimo será:

1. README.md

- Cómo ejecutar el sistema con `docker-compose`.
- Estructura del proyecto.
- Descripción de cada servicio.
- Descripción del workflow n8n y/o DAGs de Airflow.
- Variables de entorno necesarias.

2. Documentación Funcional

- Diagrama de arquitectura.
- Flujo de tareas.
- Explicación del pipeline de Fase 1 y Fase 2.
- Ejemplos de entrada y salida.
- Justificación del uso de LLM y modelo ML.
- Justificación del uso de n8n y/o Airflow.

3. Gestión de Errores

- Qué ocurre si falla un servicio.
- Qué ocurre si el LLM no responde.
- Qué ocurre si una tarea del workflow no se puede ejecutar.
- Estrategias de reintento de n8n y/o Airflow.
- Registro de errores en Postgres.
- Notificación de errores mediante n8n, si se implementa.

4. Modelo de Machine Learning

- Dataset utilizado.
- Variables de entrenamiento.
- Algoritmo elegido.
- Métricas obtenidas.
- Proceso de guardado y carga del modelo.

5. **Presentación del Proyecto** (8-10 diapositivas)

- Descripción del problema.
- Arquitectura propuesta.
- Pipeline Fase 1.
- Pipeline Fase 2.
- Ejemplo de procesamiento.
- Modelo de datos.
- Gestión de errores.
- Métricas y resultados.
- Conclusiones.
- Propuestas de mejora.

7. Desarrollo del Proyecto

1. **Planificación**

- División por servicios.
- Elección del motor de flujo: n8n, Airflow o ambos.
- Definición de workflows o DAGs.
- Definición de contratos de payload.
- Definición de prompts LLM.
- Definición de etiquetas y scores.
- Diseño de tablas y buckets.

2. **Implementación**

- Desarrollo independiente por servicio.
- Implementación del workflow en n8n y/o Airflow.
- Implementación de llamadas HTTP o tareas Python.
- Persistencia en Postgres.
- Almacenamiento de artefactos en minIO.
- Entrenamiento y guardado del modelo ML.
- Registro de ejecuciones y estados.

3. **Pruebas**

- Unitarias por servicio.
- Pruebas de integración del workflow.

- Pruebas de llamadas al LLM.
- Validación de contratos JSON.
- Validación del flujo completo Fase 1.
- Validación del flujo completo Fase 2.
- Pruebas de reintentos y errores en n8n/Airflow.

4. Evaluación del modelo

- Separación de datos de entrenamiento y prueba.
- Evaluación con métricas cuantitativas.
- Comparación entre predicción entrenada y predicción de prueba.
- Registro de valoración final.

 13 de mayo de 2026